

DL Lab 2

Binary Classification

Modules, forward propagation, backpropagation, and optimization

TA :

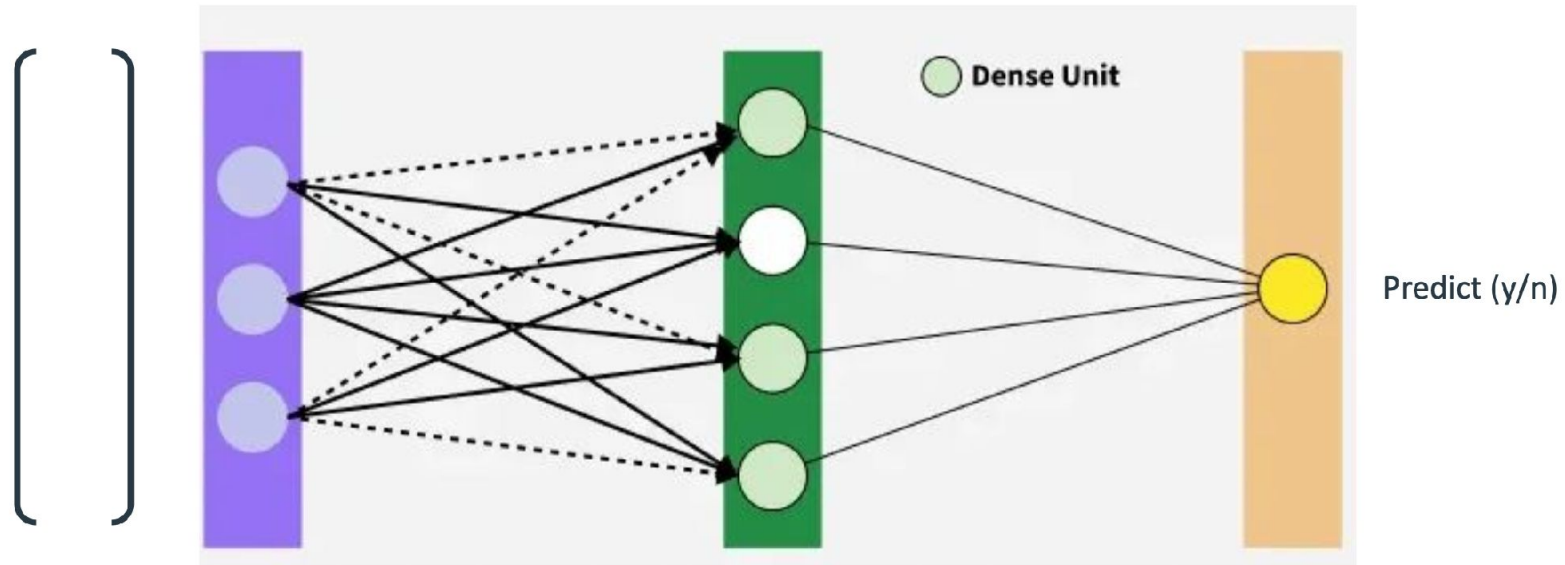
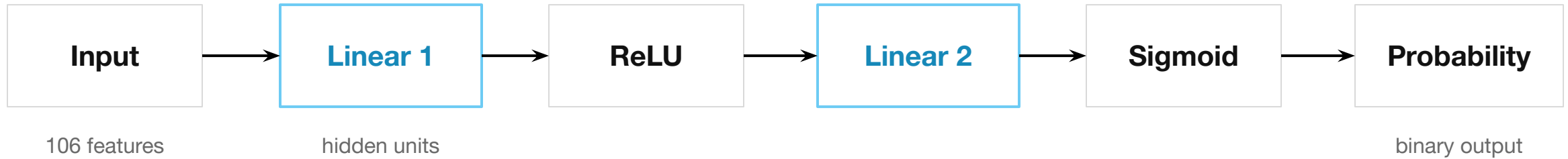
No autograd

李俊彥 jimmyli90719.ai14@nycu.edu.tw

吳仁傑 wljurse.ai14@nycu.edu.tw

The complete model

OVERVIEW



Assignment map

OVERVIEW

01	Data preparation	TODO 1
02	Forward and backward modules	TODO 2–9
03	Complete model	TODO 10–12
04	Training loop	TODO 13–15
05	Alternative update rules	TODO 16–18

Input

Each sample contains 106 processed features from the Adult Income dataset.

Label

One binary target per sample: income at or below 50K, or above 50K.

Mini-batch

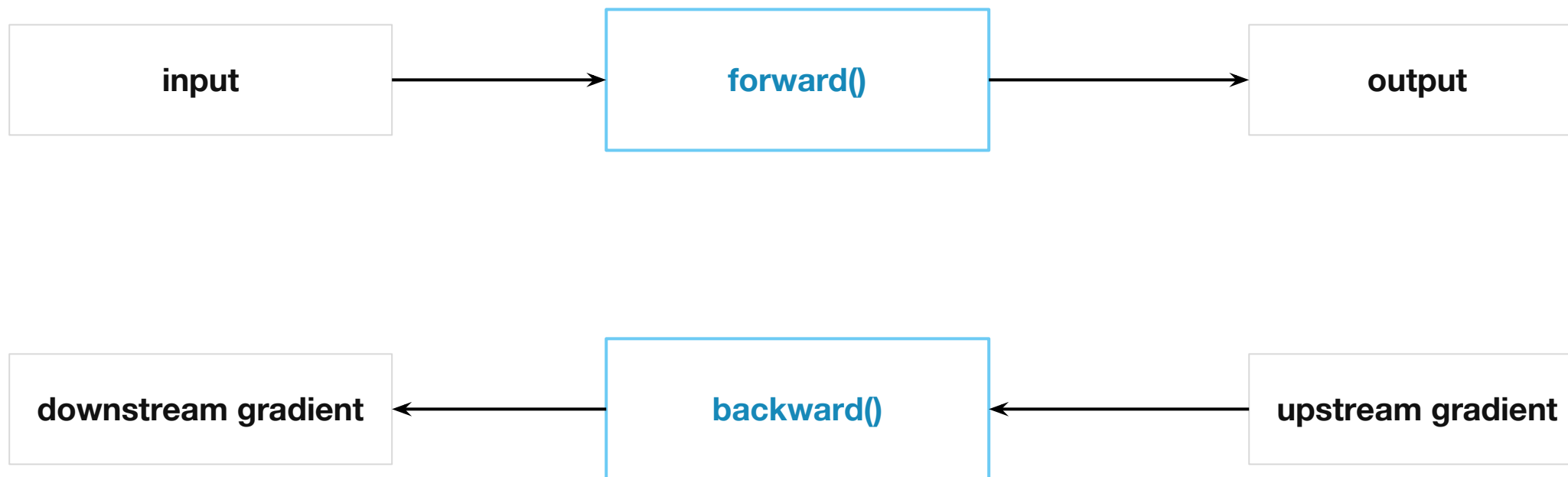
The DataLoader groups samples and labels before each training iteration.

X (B, 106)

y (B, 1)

B changes for the final batch

A shared interface for every module



Linear layer: forward

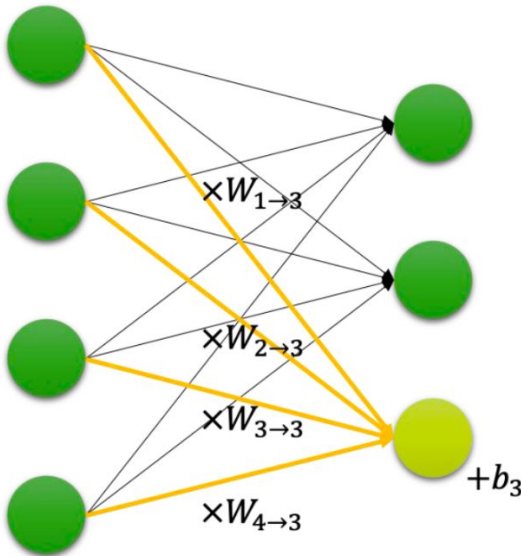
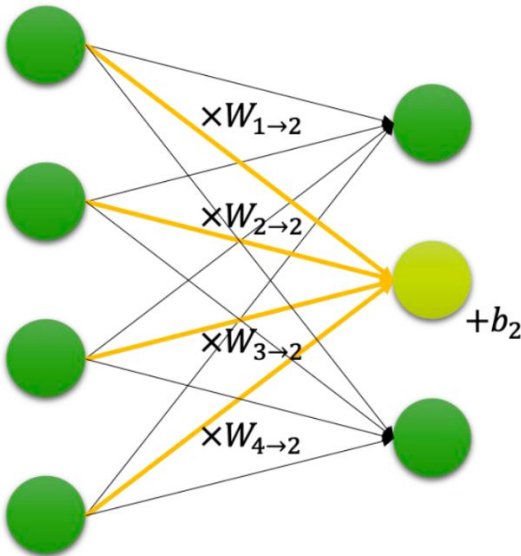
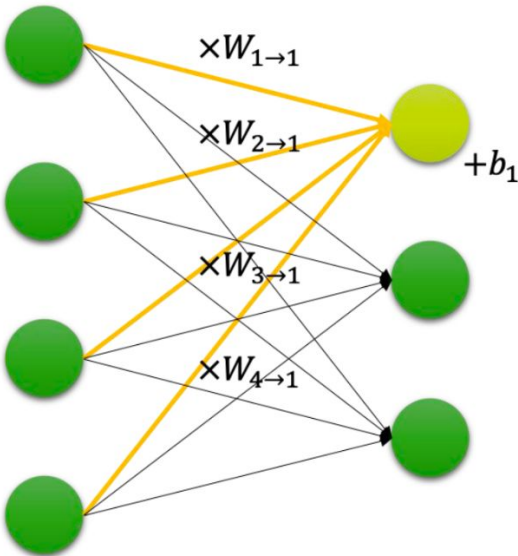
X input batch (B, D)

W trainable weights (D, H)

$$Y = XW + b$$

b trainable bias $(H,)$

Y layer output (B, H)



$$\text{ReLU}(x) = \begin{cases} x, & x > 0 \\ 0, & x \leq 0 \end{cases}$$

$$\frac{d \text{ReLU}(x)}{dx} = \begin{cases} 1, & x > 0 \\ 0, & x \leq 0 \end{cases}$$

Input

-2 -1 0 1 2

Output

0 0 0 1 2

Cache the original input. Backward uses its positive-value mask.

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

$$z = -1$$

$$0.27$$

$$z = 0$$

$$0.50$$

$$z = 2$$

$$0.88$$

Cache p for backward: $\frac{dp}{dz} = p(1 - p)$

$$p = \sigma(z)$$

$$\frac{d\sigma(z)}{dz} = \sigma(z)(1 - \sigma(z))$$

$$L = -[y \log(p) + (1 - y) \log(1 - p)]$$

$$L = -[1 \log(p) + 0 \log(1 - p)] = -\log(p)$$

When $y = 1$

The loss is $-\log(p)$. A larger p gives a smaller loss.

$$L = -[0 \log(p) + 1 \log(1 - p)] = -\log(1 - p)$$

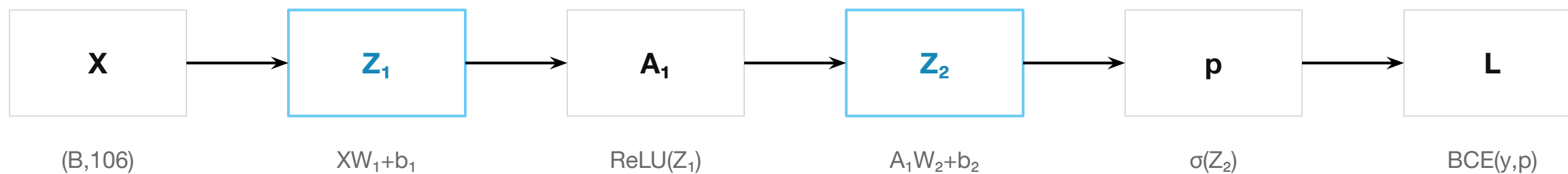
When $y = 0$

The loss is $-\log(1-p)$. A smaller p gives a smaller loss.

Mean reduction averages the sample losses across the current batch

Complete forward propagation

FORWARD



$$a = g(x) \quad L = f(a)$$

$$\frac{dL}{dx} = \frac{dL}{da} \times \frac{da}{dx}$$

$$x \rightarrow a \rightarrow L$$

DOWNSTREAM

LOCAL

UPSTREAM

$$\frac{dL}{dx}$$

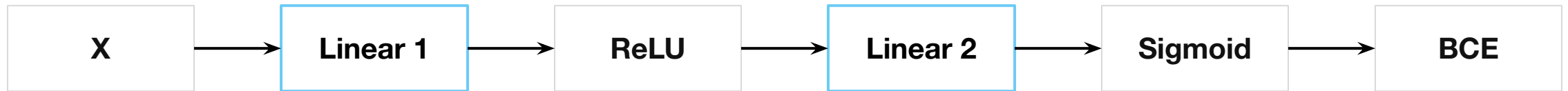
$$\frac{da}{dx}$$

$$\frac{dL}{da}$$

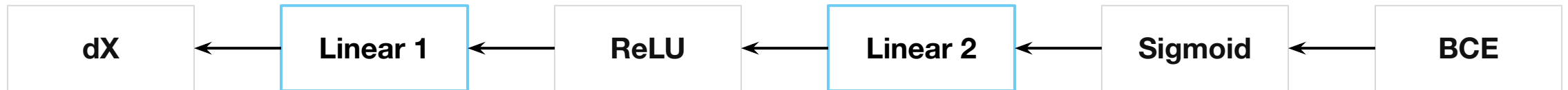
Backward runs in reverse order

BACKWARD

FORWARD



BACKWARD



BCE and Sigmoid backward

BACKWARD

BACKWARD



$$\frac{dL}{dp} = -\frac{y}{p} + \frac{1-y}{1-p}$$

BCE

$$\frac{dp}{dz} = p(1-p)$$

Sigmoid

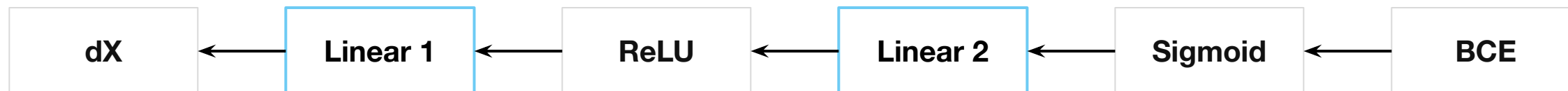
$$\frac{dL}{dz} = -y(1-p) + (1-y)p = p - y$$

For mean reduction, divide the batch gradient by B

Linear layer: backward

BACKWARD

BACKWARD



$$Y = XW + b$$

To the previous layer

$$\frac{\partial L}{\partial X} = \frac{\partial L}{\partial Y} W^T$$

$$\frac{\partial y}{\partial x} = w$$

For the weight update

$$\frac{\partial L}{\partial W} = X^T \frac{\partial L}{\partial Y}$$

$$\frac{\partial y}{\partial w} = x$$

For the bias update

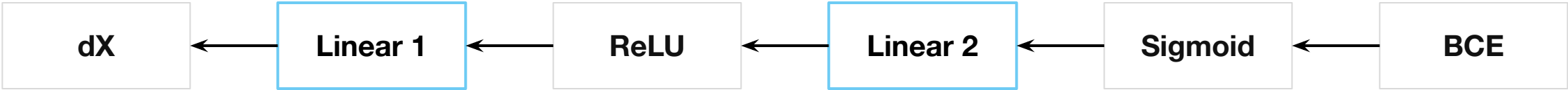
$$\frac{\partial L}{\partial b} = \sum_{\text{batch}} \frac{\partial L}{\partial Y}$$

$$\frac{\partial y}{\partial b} = 1$$

ReLU backward

BACKWARD

BACKWARD

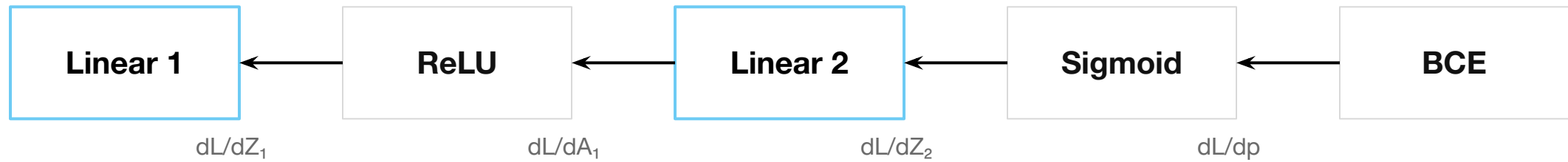


$$dX = \text{upstream gradient} \times (\text{original input} > 0)$$

Original input	-2	0	3
Upstream gradient	5	5	5
Downstream gradient	0	0	5

Complete backward propagation

BACKWARD



Linear 1 stores dW_1 and db_1

Linear 2 stores dW_2 and db_2

chain rule

$$\frac{dL}{dx} = \frac{dL}{da} \times \frac{da}{dx}$$

One training iteration

TRAINING

- | | | |
|----|---------------------|-------------------------------|
| 01 | zero_grad | clear parameter gradients |
| 02 | forward | compute prediction |
| 03 | compute_loss | compare prediction with label |
| 04 | backward | compute gradients |
| 05 | optimize | update W and b |

Stochastic gradient descent

$$W \leftarrow W - \eta \frac{\partial L}{\partial W}$$

Current gradient

SGD uses the gradients produced by the latest mini-batch.

$$b \leftarrow b - \eta \frac{\partial L}{\partial b}$$

Learning rate η

The learning rate controls the size of every parameter update.

Weight decay adds a preference for smaller weights

$$L_{\text{total}} = L_{\text{data}} + \lambda \sum W^2$$

$$dW_{\text{total}} = dW_{\text{data}} + 2\lambda W$$

$$W_{\text{new}} = W - \eta(dW_{\text{data}} + 2\lambda W)$$

$$W_{\text{new}} = (1 - 2\eta\lambda)W - \eta dW_{\text{data}}$$

$$v_t = \mu v_{t-1} + g_t$$

$$\theta_t = \theta_{t-1} - \eta v_t$$

v

velocity stored across mini-batches

μ

momentum coefficient

g

current parameter gradient

Momentum μ	Retained after 1 step	Retained after 2 steps	Retained after 3 steps
0.5	0.5	0.25	0.125
0.9	0.9	0.81	0.729

m

Moving average of gradients

v

Moving average of squared gradients

t

Number of optimizer updates

adaptive gradient update

decoupled weight decay

1. Deadline: 10/4 23:59
2. Only ipynb file is acceptable. Or you will get 0 score.
3. Please strictly follow the provided code framework.
4. Do not copy others code.
5. Pytorch Auto-grad is not allowed.
6. Hand on Filename : HW2_SID.ipynb
e.g HW2_313831019.ipynb

$$Y = XW + b$$

X input batch **(B, D)**

W trainable weights **(D, H)**

b trainable bias **(H,)**

Y layer output **(B, H)**

Update rules compared

Method	Current gradient	Past direction	Adaptive step	Weight decay
SGD	Yes	No	No	No
Weight decay	Yes	No	No	Yes
Momentum	Yes	Yes	No	If implemented
AdamW	Yes	Yes	Yes	Decoupled